Bài 2.7 – KNN (K-Nearest Neighbors)

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *So sánh ví von*

🎯 Sau bài này bạn sẽ

- Hiểu KNN, thuật toán ML **đơn giản nhất**: không "học" gì, chỉ "nhớ" và "hỏi hàng xóm".
- Tự tính tay một dự đoán KNN.
- Thấy tận mắt vì sao KNN **bắt buộc chuẩn hóa** dữ liệu.
- Chọn K và hiểu K liên quan đến bias–variance (bài 2.6) thế nào.
- Biết điểm yếu chết người của KNN: **lời nguyền số chiều**.

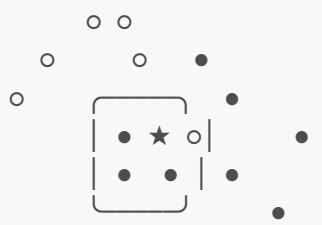
---

1. So sánh ví von: "Gần mực thì đen, gần đèn thì rạng"

Bạn chuyển đến một khu phố mới và muốn đoán: "Nhà mình đáng giá bao nhiêu?"

Cách đơn giản nhất: hỏi giá **5 căn nhà gần nhà bạn nhất** có diện tích tương tự, rồi lấy trung bình. Không cần công thức, không cần hệ số.

Đó chính là **KNN**. Câu tục ngữ "Hãy cho tôi biết bạn chơi với ai, tôi sẽ nói bạn là người thế nào" là ý tưởng của thuật toán này.



```
Điểm mới ★ cần phân loại.
Lấy K = 5 hàng xóm gần nhất (trong vòng tròn):
   3 điểm ● và 2 điểm ○
  → Bỏ phiếu: ● thắng 3–2
→ ★ được dự đoán là ●
```

---

2. Thuật toán

2.1. Khái niệm

- **KNN – K-Nearest Neighbors (K láng giềng gần nhất):** Để dự đoán một điểm mới, tìm **K điểm gần nhất** trong dữ liệu train, rồi:
  - **Phân loại:** bỏ phiếu, lớp nào nhiều phiếu nhất thắng.
  - **Hồi quy:** lấy trung bình giá trị của K hàng xóm.
- **"Gần"** thường đo bằng **khoảng cách Euclid** (bài 1.8).

2.2. KNN là "học lười" (lazy learning)

	Linear/Logistic Regression	**KNN**
Lúc `fit`	Học ra hệ số w, b (tốn công)	**Chỉ lưu lại** toàn bộ dữ liệu (gần như tức thì)
Lúc `predict`	Tính w·x + b (rất nhanh)	Đo khoảng cách tới **mọi** điểm train (chậm khi dữ liệu lớn)
Mô hình "nhớ" gì	Vài con số	Toàn bộ dữ liệu train

2.3. Tính tay

Một app đánh giá quán ăn muốn đoán: quán mới có khiến khách **quay lại** không? Dữ liệu 5 quán cũ: [giá trung bình một món (nghìn đồng), khoảng cách tới trung tâm (km)].

Quán	**Giá món**	**Km**	**Khách quay lại?**
A	30	1	Có
B	45	2	Có
C	35	5	Không
D	120	3	Không
E	150	1	Có

Quán mới: giá **40**, cách **2 km**. Dùng K = 3.

Bước 1: tính khoảng cách tới từng quán.

- B: $\sqrt{[(45-40)^2 + (2-2)^2]} = \sqrt{25} =$ **5,00**
- C: $\sqrt{[(35-40)^2 + (5-2)^2]} = \sqrt{(25+9)} = \sqrt{34} \approx$ **5,83**
- A: $\sqrt{[(30-40)^2 + (1-2)^2]} = \sqrt{101} \approx$ **10,05**
- D: $\sqrt{[80^2 + 1^2]} \approx$ **80,01**
- E: $\sqrt{[110^2 + 1^2]} \approx$ **110,00**

Bước 2: 3 quán gần nhất: B (Có), C (Không), A (Có). **Bước 3:** Bỏ phiếu: Có 2 – Không 1 → **dự đoán: Có**.

```
import numpy as np
import pandas as pd

quan = pd.DataFrame({"ten": ["A", "B", "C", "D", "E"],
                     "gia_mon": [30, 45, 35, 120, 150],
                     "km": [1, 2, 5, 3, 1],
                     "khach_quay_lai": ["Có", "Có", "Không", "Không", "Có"]})
moi = np.array([40, 2])
quan["khoang_cach"] = np.sqrt((quan["gia_mon"] - moi[0]) ** 2 + (quan["km"] -
moi[1]) ** 2).round(2)
print(quan.sort_values("khoang_cach"))
```

Kết quả mẫu:

```
  ten  gia_mon  km khach_quay_lai  khoang_cach
1   B       45   2             Có         5.00
2   C       35   5          Không         5.83
0   A       30   1             Có        10.05
3   D      120   3          Không        80.01
4   E      150   1             Có       110.00
```

Bạn có thấy vấn đề không? Khoảng cách gần như chỉ phụ thuộc vào **giá món** (chênh hàng chục, hàng trăm), cột **km** (chênh vài đơn vị) gần như không có tiếng nói. Cùng vấn đề ở bài 1.11. Mục 3 sẽ cho thấy hậu quả trên dữ liệu thật.

---

3. Bắt buộc chuẩn hóa: thí nghiệm với dữ liệu rượu vang

3.1. Dữ liệu

Bộ dữ liệu **Wine** (có sẵn trong Scikit-learn): 178 mẫu rượu vang từ 3 giống nho, 13 chỉ số hóa học. Nhiệm vụ: đoán giống nho (lớp 0, 1, 2) từ các chỉ số.

```
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)          # as_frame=True: trả về DataFrame
X, y = wine.data, wine.target
print(X.shape, y.value_counts().sort_index().tolist())
print(X[["alcohol", "malic_acid", "magnesium", "proline"]].describe().loc[["mean",
"std", "min", "max"]].round(2))
```

Kết quả mẫu:

```
(178, 13) [59, 71, 48]
      alcohol  malic_acid  magnesium  proline
```

```
mean    13.00        2.34      99.74   746.89
std      0.81        1.12      14.28   314.91
min     11.03        0.74      70.00   278.00
max     14.83        5.80     162.00  1680.00
```

Cột `proline` có giá trị hàng trăm đến hàng nghìn. Cột `malic_acid` chỉ từ 0,7 đến 5,8. Không chuẩn hóa, khoảng cách gần như chỉ do `proline` quyết định.

3.2. So sánh

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)

k5 = KNeighborsClassifier(n_neighbors=5).fit(Xtr, ytr)
print("Không chuẩn hóa:", round(k5.score(Xte, yte), 3))

# make_pipeline: chuẩn hóa rồi KNN, scaler tự fit chỉ trên train (sẽ học kỹ ở bài
2.17)
k5s = make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=5)).fit(Xtr, ytr)
print("Có chuẩn hóa:", round(k5s.score(Xte, yte), 3))
```

Kết quả mẫu:

```
Không chuẩn hóa: 0.722
Có chuẩn hóa: 0.944
```

Chỉ thêm một bước chuẩn hóa, độ chính xác tăng từ **72% lên 94%**. Với KNN, chuẩn hóa không phải "nên làm" mà là **bắt buộc**.

3.3. Nhìn vào hàng xóm

```
mau = Xte.iloc[[0]]                                    # mẫu đầu tiên của test
(giữ dạng bảng)
khoang_cach, vi_tri = k5s[-1].kneighbors(k5s[0].transform(mau))   # k5s[0]:
scaler; k5s[-1]: KNN
print(khoang_cach.round(2), ytr.iloc[vi_tri[0]].tolist(),         # khoảng cách và
nhãn 5 hàng xóm
      k5s.predict_proba(mau), yte.iloc[0])                        # xác suất dự
đoán và nhãn thật
```

Kết quả mẫu:

```
[[2.03 2.06 2.21 2.56 2.63]] [0, 0, 0, 0, 0] [[1. 0. 0.]] 0
```

Cả 5 hàng xóm đều thuộc lớp 0 → `predict_proba` = 100% lớp 0. Với KNN, "xác suất" chỉ đơn giản là **tỉ lệ phiếu bầu**.

---

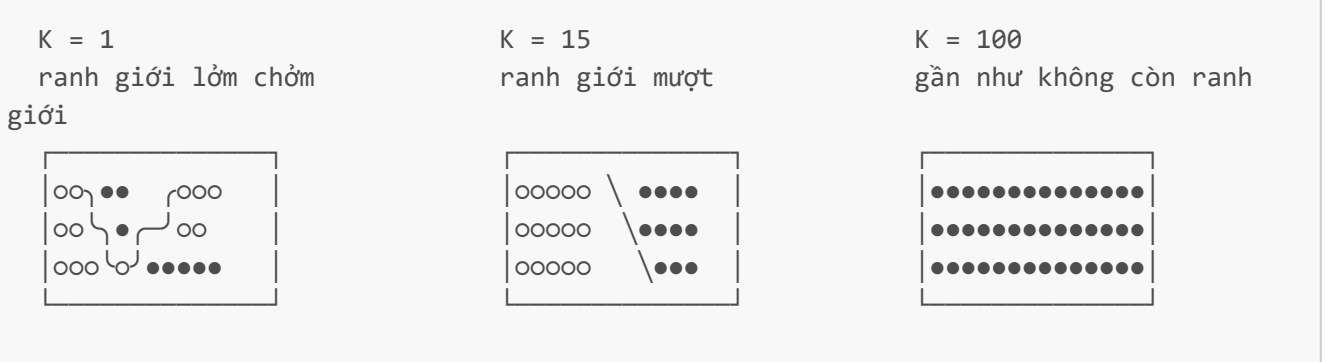
4. Chọn K

```
for k in [1, 3, 5, 11, 21, 51, 101]:
    m = make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=k)).fit(Xtr, ytr)
    print(f"K = {k:>3}: accuracy train = {m.score(Xtr, ytr):.3f} | test =
{m.score(Xte, yte):.3f}")
```

Kết quả mẫu:

```
K =   1: accuracy train = 1.000 | test = 0.963
K =   3: accuracy train = 0.984 | test = 0.944
K =   5: accuracy train = 0.984 | test = 0.944
K =  11: accuracy train = 0.976 | test = 0.963
K =  21: accuracy train = 0.968 | test = 1.000
K =  51: accuracy train = 0.976 | test = 0.926
K = 101: accuracy train = 0.444 | test = 0.426
```

K	Hành vi	Liên hệ bài 2.6
K = 1	Train luôn 100% (mỗi điểm là hàng xóm gần nhất của chính nó). Ranh giới lởm chởm, nhạy với nhiễu	**Variance cao** (dễ overfit)
K vừa phải	Ranh giới mượt, ổn định	Cân bằng
K rất lớn (101 trong 124 điểm train)	Gần như hỏi **cả lớp học**: luôn đoán lớp đông nhất. Accuracy rơi xuống 43%	**Bias cao** (underfit)



Lưu ý trung thực: Tập test chỉ có **54 mẫu**, nên mỗi mẫu đoán sai làm accuracy thay đổi gần 2%. Chênh lệch giữa K = 1, 5, 11, 21 phần lớn là **may rủi**, không phải khác biệt thật. Đừng chọn K = 21 chỉ vì nó đạt 100% trên tập test nhỏ này. Cách chọn K đáng tin: cross-validation (bài 2.17–2.18).

Mẹo thực tế:

- Bắt đầu với K khoảng 5–15.
- Phân loại 2 lớp: chọn K **lẻ** để tránh hòa phiếu.

---

5. KNN cho hồi quy: giá nhà theo vị trí

Ý tưởng tự nhiên: giá một khu ≈ giá trung bình của các khu **gần nó nhất trên bản đồ**.

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error

df = nap_du_lieu_nha()       # ô chuẩn bị dữ liệu ở bài 2.1
Xtr, Xte, ytr, yte = train_test_split(df[["kinh_do", "vi_do"]], df["gia_nha"],
test_size=0.2, random_state=42)
for k in [1, 5, 10, 20, 50]:
    m = KNeighborsRegressor(n_neighbors=k).fit(Xtr, ytr)
    print(f"K = {k:>2}: MAE = {mean_absolute_error(yte, m.predict(Xte)):.1f}")

# weights="distance": hàng xóm càng gần, "phiếu" càng có trọng lượng
mw = KNeighborsRegressor(n_neighbors=10, weights="distance").fit(Xtr, ytr)
print("K = 10, trọng số theo khoảng cách:", round(mean_absolute_error(yte,
mw.predict(Xte)), 1))
```

Kết quả mẫu:

```
K =  1: MAE = 35.5
K =  5: MAE = 30.9
K = 10: MAE = 31.2
K = 20: MAE = 33.5
K = 50: MAE = 38.0
K = 10, trọng số theo khoảng cách: 30.9
```

Bất ngờ lớn: Chỉ dùng **2 cột kinh độ, vĩ độ**, KNN đạt MAE **30,9**, tốt hơn hẳn Linear Regression với 6 đặc trưng (48,7, bài 2.3) và cả Ridge đa thức. Vì sao? Giá nhà phụ thuộc vị trí theo kiểu **"từng khu một"** (khu này đắt, khu bên cạnh cũng đắt), một quy luật **cục bộ** mà đường thẳng không thể diễn tả, nhưng "hỏi hàng xóm" thì nắm được ngay.

(Ở đây không cần chuẩn hóa vì kinh độ và vĩ độ cùng đơn vị "độ".)

---

6. Điểm yếu: lời nguyền số chiều

Thêm cả 8 đặc trưng (đã chuẩn hóa) có giúp KNN tốt hơn không?

```
cot = ["thu_nhap", "km_tp_lon", "phong_moi_ho", "ti_le_phong_ngu",
       "nguoi_moi_ho", "tuoi_nha", "kinh_do", "vi_do"]
Xtr, Xte, ytr, yte = train_test_split(df[cot], df["gia_nha"], test_size=0.2,
random_state=42)
m = make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=10)).fit(Xtr,
ytr)
print("8 đặc trưng:", round(mean_absolute_error(yte, m.predict(Xte)), 1))
```

Kết quả mẫu:

```
8 đặc trưng: 36.3
```

Tệ hơn dùng 2 cột vị trí (30,9)! Thêm thông tin mà mô hình lại kém đi. Vì KNN coi **mọi cột quan trọng như nhau** khi tính khoảng cách. Các cột ít liên quan (như `nguoi_moi_ho`) làm "nhiễu" khái niệm "gần".

Tệ hơn nữa, khi số chiều rất lớn, **mọi điểm đều xa nhau gần như như nhau**:

```
rng = np.random.default_rng(0)
for so_chieu in [2, 10, 100, 1000]:
    P = rng.uniform(size=(500, so_chieu))        # 500 điểm ngẫu nhiên
    q = rng.uniform(size=so_chieu)               # 1 điểm cần tìm hàng xóm
    kc = np.linalg.norm(P - q, axis=1)           # khoảng cách tới 500 điểm
    print(so_chieu, "chiều: gần nhất / xa nhất =", round(kc.min() / kc.max(), 3))
```

Kết quả mẫu:

```
2 chiều: gần nhất / xa nhất = 0.024
10 chiều: gần nhất / xa nhất = 0.177
100 chiều: gần nhất / xa nhất = 0.737
1000 chiều: gần nhất / xa nhất = 0.904
```

Ở 2 chiều, điểm gần nhất gần **gấp 40 lần** điểm xa nhất. Ở 1.000 chiều, hàng xóm gần nhất cũng xa gần bằng điểm xa nhất (90%). Khi đó "hàng xóm gần nhất" **mất hết ý nghĩa**. Đây là **Curse of Dimensionality (Lời nguyền số chiều)**.

Cách chữa: chọn lọc đặc trưng, hoặc giảm chiều bằng PCA (bài 2.15) trước khi dùng KNN.

---

7. Ưu, nhược điểm

Ưu điểm	Nhược điểm
Cực kỳ dễ hiểu, dễ giải thích ("vì 5 khách giống bạn nhất đều...")	**Bắt buộc** chuẩn hóa
Không giả định hình dạng quy luật, nắm được quy luật cục bộ	Dự đoán chậm khi dữ liệu train lớn (phải đo khoảng cách tới mọi điểm)
Thêm dữ liệu mới chỉ cần "lưu thêm", không cần huấn luyện lại	Kém khi nhiều chiều hoặc nhiều đặc trưng vô ích
Dùng được cho cả phân loại và hồi quy	Phải lưu toàn bộ dữ liệu train

Ứng dụng thực tế: hệ gợi ý "khách hàng tương tự" (bài 1.8), tìm sản phẩm giống nhau, điền dữ liệu thiếu (`KNNImputer`), và là nền tảng của **tìm kiếm vector** trong chatbot RAG (sẽ học ở bài NLP4.2).

---

8. Bài tập

1. Ở ví dụ quán ăn (mục 2.3), chuẩn hóa hai cột bằng min-max **tính tay** (giá: min 30, max 150; km: min 1, max 5), cũng chuẩn hóa quán mới [40, 2] bằng cùng min, max. Tính lại khoảng cách. Kết quả bỏ phiếu với K = 3 có đổi không?
2. Ở mục 5, thử K = 5 với `weights="distance"`. MAE có tốt hơn 30,9 không?
3. Ở mục 6, chỉ dùng 3 cột: `kinh_do`, `vi_do`, `thu_nhap` (có chuẩn hóa), K = 10. MAE so với 30,9 và 36,3 thế nào? Rút ra bài học gì?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Giá chuẩn hóa = (giá − 30) / 120; km chuẩn hóa = (km − 1) / 4. Quán mới: [(40−30)/120; (2−1)/4] = [0,083; 0,25].

   - A [0; 0]: $\sqrt{(0{,}083^2 + 0{,}25^2)} \approx 0{,}263$
   - B [0,125; 0,25]: $\sqrt{(0{,}042^2 + 0)} \approx$ **0,042**
   - C [0,042; 1]: $\sqrt{(0{,}042^2 + 0{,}75^2)} \approx 0{,}751$
   - D [0,75; 0,5]: $\sqrt{(0{,}667^2 + 0{,}25^2)} \approx 0{,}712$
   - E [1; 0]: $\sqrt{(0{,}917^2 + 0{,}25^2)} \approx 0{,}950$

   3 gần nhất: **B (Có), A (Có), D (Không)** → vẫn dự đoán **Có**, nhưng hàng xóm đã đổi: C (cách xa 5 km) bị loại, D (giá đắt nhưng gần) được đưa vào. Giờ cả hai cột đều có tiếng nói.

2. Code:

```
Xtr2, Xte2, ytr2, yte2 = train_test_split(df[["kinh_do", "vi_do"]], df["gia_nha"],
test_size=0.2, random_state=42)
m = KNeighborsRegressor(n_neighbors=5, weights="distance").fit(Xtr2, ytr2)
print(round(mean_absolute_error(yte2, m.predict(Xte2)), 1))
```

So sánh với 30,9 và ghi lại. Khác biệt thường rất nhỏ: với dữ liệu dày đặc, trọng số theo khoảng cách không thay đổi nhiều.

3. Code:

```
cot3 = ["kinh_do", "vi_do", "thu_nhap"]
Xtr3, Xte3, ytr3, yte3 = train_test_split(df[cot3], df["gia_nha"], test_size=0.2,
random_state=42)
m = make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors=10)).fit(Xtr3,
ytr3)
print(round(mean_absolute_error(yte3, m.predict(Xte3)), 1))
```

Thêm **đúng một** đặc trưng mạnh (thu nhập) vào vị trí thường giúp ích, trong khi thêm **nhiều** đặc trưng yếu lại làm hại. Bài học: với KNN, **chất lượng đặc trưng quan trọng hơn số lượng**.

---

9. Flashcard

Mặt trước	Mặt sau
KNN dự đoán thế nào?	Tìm K điểm train gần nhất; phân loại: bỏ phiếu; hồi quy: lấy trung bình.
Vì sao KNN gọi là "học lười"?	`fit` chỉ lưu dữ liệu; mọi tính toán dồn vào lúc `predict`.
KNN có cần chuẩn hóa?	**Bắt buộc**, vì dựa trên khoảng cách.
K = 1 dẫn tới gì?	Variance cao, dễ overfit, train luôn 100%.
K rất lớn dẫn tới gì?	Bias cao, underfit, tiến về đoán lớp đông nhất.
Phân loại 2 lớp nên chọn K thế nào?	Số lẻ, để tránh hòa phiếu.
Lời nguyền số chiều?	Nhiều chiều → mọi điểm xa nhau gần như như nhau → "gần nhất" vô nghĩa.
`weights="distance"` làm gì?	Hàng xóm gần hơn có phiếu nặng hơn.
`predict_proba` của KNN là gì?	Tỉ lệ phiếu bầu của K hàng xóm.

Bài tiếp theo: 2.8 – Naive Bayes: lọc tin nhắn spam.